DESENVOLVIMENTO MOBILE

Desenvolvimento Android: da aplicação ao hardware

Aula 04 - Sistema Android, Android Runtime, Application Framework, acesso ao hardware e arquitetura em camadas

O desenvolvimento de aplicações para dispositivos móveis envolve mais do que a construção da interface exibida na tela. Ao executar um aplicativo Android, diferentes componentes do sistema operacional trabalham de maneira integrada para executar o código da aplicação, disponibilizar serviços, administrar recursos computacionais e permitir a comunicação com componentes físicos do dispositivo.

O Android pode ser compreendido como uma plataforma de software organizada em diferentes níveis de abstração. Essa organização permite que o desenvolvedor construa aplicações utilizando recursos de alto nível sem precisar conhecer detalhadamente o funcionamento eletrônico de cada modelo de smartphone.

Maior nível de abstração	**APLICAÇÕES**
	APPLICATION FRAMEWORK
	ANDROID RUNTIME + BIBLIOTECAS
	CAMADAS DE ABSTRAÇÃO / HAL
Menor nível de abstração	**LINUX KERNEL**
	HARDWARE

1. Linux Kernel: a base do sistema Android

Na base da arquitetura Android encontra-se o Linux Kernel. O kernel é o núcleo de um sistema operacional e está relacionado ao gerenciamento dos recursos fundamentais necessários para que programas sejam executados e para que o software se comunique com componentes físicos do dispositivo.

No Android, o kernel exerce funções relacionadas ao gerenciamento de processos e memória, segurança, energia, comunicação entre processos, dispositivos e drivers de hardware. Por estar próximo do hardware, ele participa de tarefas fundamentais que normalmente permanecem invisíveis para o desenvolvedor de aplicações.

1.1 Drivers e componentes físicos

Um driver é um componente de software que permite ao sistema operacional comunicar-se com determinado dispositivo de hardware. Smartphones possuem câmera, microfone, alto-falante, Bluetooth, Wi-Fi, tela, GPS e diversos sensores.

Esses componentes podem variar significativamente entre fabricantes e modelos. O desenvolvedor de uma aplicação não deveria precisar conhecer as características técnicas específicas de cada câmera ou sensor disponível no mercado. A arquitetura Android utiliza camadas e interfaces para reduzir essa complexidade.

2. Android Runtime

Acima dos componentes mais próximos do sistema encontra-se um elemento fundamental para a execução das aplicações: o Android Runtime. Nas versões modernas do Android, esse ambiente é conhecido como ART - Android Runtime.

De forma introdutória, o Android Runtime pode ser compreendido como parte da infraestrutura responsável pela execução das aplicações Android. Existe uma diferença entre o código-fonte produzido pelo programador e aquilo que efetivamente é executado pelo dispositivo. O código passa por processos de compilação e preparação antes de ser executado no ambiente Android.

2.1 Kotlin e Android Runtime não são a mesma coisa

É importante distinguir os conceitos: Kotlin é uma linguagem de programação utilizada para escrever código; o Android Runtime faz parte da infraestrutura utilizada pelo Android para executar as aplicações. Portanto, linguagem de programação e ambiente de execução exercem papéis diferentes no ecossistema.

3. Application Framework

O desenvolvedor Android normalmente trabalha em um nível consideravelmente mais alto do que o Linux Kernel. Um dos níveis mais importantes é o Application Framework, ou framework de aplicações.

Essa camada disponibiliza serviços, componentes e APIs - Application Programming Interfaces - que podem ser utilizados pelos aplicativos. Uma API fornece uma maneira estruturada para que um programa utilize funcionalidades disponibilizadas pela plataforma.

Em vez de desenvolver todas as funcionalidades desde o início, uma aplicação pode utilizar recursos oferecidos pelo Android para apresentar telas, acessar recursos, utilizar localização, emitir notificações, interagir com outras partes do sistema e solicitar acesso a determinados recursos do dispositivo.

4. O conceito de abstração

Para compreender a arquitetura Android, é fundamental entender o conceito de abstração. Na Ciência da Computação, abstração permite esconder determinados detalhes de implementação e apresentar ao programador uma interface mais simples para utilizar uma funcionalidade.

Considere uma aplicação que precisa utilizar a câmera. O desenvolvedor está interessado em abrir a câmera, capturar uma imagem e receber o resultado. Normalmente ele não precisa controlar diretamente sinais elétricos, registradores ou detalhes específicos do sensor utilizado naquele smartphone.

5. Acesso ao hardware

Os smartphones possuem recursos físicos como câmera, GPS, acelerômetro, giroscópio, microfone, Bluetooth, NFC, leitor biométrico, sensores de proximidade e conectividade Wi-Fi. Seria inviável exigir que cada desenvolvedor conhecesse detalhadamente todas as implementações desses componentes.

A plataforma Android fornece diferentes níveis de abstração. Assim, uma aplicação normalmente utiliza APIs disponibilizadas pela plataforma, enquanto outras camadas cuidam da comunicação necessária com componentes de hardware.

6. Hardware Abstraction Layer - HAL

Um conceito importante nessa comunicação é a Hardware Abstraction Layer (HAL), ou Camada de Abstração de Hardware. A HAL ajuda a estabelecer interfaces entre partes do sistema Android e implementações específicas de hardware.

Imagine smartphones de fabricantes diferentes utilizando câmeras distintas. Para as camadas superiores do sistema, é desejável que exista uma maneira consistente de solicitar funcionalidades, independentemente dos detalhes físicos de cada implementação. Essa abstração é especialmente importante no Android devido à grande diversidade de fabricantes e configurações de hardware.

7. Exemplo: uma aplicação utilizando a câmera

Considere um aplicativo que precisa capturar uma fotografia. Para o usuário, o processo parece simples: ele toca em um botão, a câmera é aberta e a fotografia é capturada. Por trás dessa ação, entretanto, diferentes componentes da plataforma participam do processo.

Aplicação	solicita a câmera
Application Framework / APIs	fornece serviços da plataforma
HAL	abstrai a implementação de hardware
Kernel / Drivers	realiza comunicação de baixo nível
Câmera	componente físico

Observação: esse fluxo é uma representação didática e simplificada. O caminho real possui detalhes técnicos adicionais.

8. Por que o Android utiliza camadas?

Uma arquitetura organizada em camadas favorece a separação de responsabilidades. Cada nível possui funções diferentes, o que ajuda a organizar a complexidade do sistema.

Outro benefício é a abstração. O desenvolvedor não precisa conhecer todos os detalhes das camadas inferiores para criar uma aplicação. Isso também contribui para a portabilidade entre dispositivos, aspecto especialmente relevante em um ecossistema com aparelhos de diferentes fabricantes e configurações.

9. Relacionando a arquitetura ao primeiro aplicativo Android

Ao desenvolver um primeiro aplicativo no Android Studio, como um Hello World, grande parte dessa arquitetura permanece invisível para o estudante. Um comando simples de interface depende de uma infraestrutura muito maior para ser compilado, executado e apresentado no dispositivo.

Mesmo uma aplicação extremamente simples depende de código, APIs, runtime, serviços da plataforma, kernel e hardware. O objetivo da arquitetura é justamente permitir que o desenvolvedor não precise controlar toda essa complexidade diretamente.

10. Android Studio, SDK e plataforma Android

Três elementos frequentemente confundidos pelos estudantes devem ser diferenciados. O Android Studio é o ambiente integrado de desenvolvimento (IDE), onde o desenvolvedor escreve código, organiza projetos, executa aplicações e utiliza ferramentas de desenvolvimento.

O Android SDK é o Software Development Kit: um conjunto de ferramentas, APIs e componentes utilizados no desenvolvimento Android. Já a plataforma Android é o ambiente no qual as aplicações são executadas, composto por diferentes camadas e componentes.

11. Do código ao hardware

Uma das principais conclusões sobre a arquitetura Android é que existe uma grande distância conceitual entre o código escrito pelo programador e a operação física realizada pelo dispositivo. Quando o desenvolvedor utiliza uma API Android, ele trabalha em um alto nível de abstração.

Nas camadas inferiores existem mecanismos responsáveis por transformar essas solicitações em operações compatíveis com o sistema e, quando necessário, com o hardware. Quanto mais próximo da aplicação, maior tende a ser o nível de abstração disponível ao desenvolvedor; quanto mais próximo do hardware, maior é a relação com aspectos internos do sistema operacional, drivers e componentes físicos.

12. Considerações finais

Compreender a arquitetura Android permite que o estudante deixe de enxergar o desenvolvimento mobile apenas como a construção de telas e botões. Uma aplicação Android faz parte de um ecossistema composto por diferentes componentes que trabalham de maneira integrada.

O Linux Kernel fornece a base para o gerenciamento dos recursos fundamentais e para a interação de baixo nível com dispositivos. O Android Runtime participa da infraestrutura de execução das aplicações. O Application Framework disponibiliza APIs e serviços de alto nível, enquanto camadas de abstração, incluindo a HAL, ajudam a reduzir a dependência entre aplicações e implementações específicas de hardware.

Compreender essas relações é fundamental para avançar no desenvolvimento Android, pois permite visualizar a arquitetura existente por trás de ações aparentemente simples realizadas por uma aplicação.

Referência bibliográfica

DEITEL, Paul; DEITEL, Harvey; WALD, Alexander. Android 6 para programadores: uma abordagem baseada em aplicativos. 3. ed. Porto Alegre: Bookman/Grupo A, 2016.

Material didático elaborado para a Aula 04 de Desenvolvimento Mobile - CEUB. A referência foi utilizada como apoio bibliográfico; não foram reproduzidos trechos extensos da obra.